

Discoverable – PWAs apps are identifiable as ‘applications’ thanks to W3C manifests and the scope of the service worker registration, which allows search engines to find them.

Re-engageable – They make re-engagement simple through features like push notifications.

Installable – They permit users to ‘keep’ apps they find most useful on their home screen without the hassle of an app store.

Linkable – PWAs are simply shared via URL and do not require complex installation.

Why progressive Web apps?

A study has shown that, usually, an app loses 20 per cent of its users for every extra step between the user’s first contact with the app and the user starting to use it. A user must first find the app in an app store, download it, install it and then, finally, open it. When users find progressive Web apps, they will be able to immediately start using these, eliminating the downloading and installation stages. When they return to the app, they will be encouraged to install it and to upgrade it to a full-screen experience.

However, a native app is definitely not all bad. Mobile applications with push notifications get up to three times more retention than their counterparts without push, and a user is three times more likely to reopen a mobile application than a website. In addition, a well-designed mobile

application consumes less data and is much faster because some resources exist on the device.

A PWA takes advantage of a mobile app’s characteristics, leading to improved user retention and performance, without the difficulties involved in maintaining a mobile application.

When should you build a progressive Web app?

PWAs are usually recommended for applications that you expect users to return to frequently. Flipkart uses a progressive Web app for its popular e-commerce platform, Flipkart Lite, and Switzerland based SBB uses one for its online check-in process, allowing users to access their tickets without an Internet connection.

When assessing whether your next application should be a PWA, a website or a native mobile application, first identify your users and their most important actions. Being ‘progressive’, a PWA works in all browsers and the experience is enhanced whenever the user’s browser is updated with new, improved features and APIs.

Thus, there is no comparison between the user experience with a PWA and a traditional website. However, you may have to choose what functionality to support offline, and you will have to assist navigation (remember that in

standalone mode, the user does not have access to the back button). If your website already has an application-like interface, applying the concepts of PWAs will only make it better.

If certain features are definitely required for critical user actions but are not yet available due to a lack of cross-browser support, then a native mobile application might be the better option, guaranteeing a similar experience for all users.

Does a PWA have any drawbacks?

The major drawback of PWAs is that they require more battery power than the conventional websites that’re visited on a mobile Web browser. This has to do with all the extra capabilities that a PWA has, like the interface and the service worker.

The second drawback is that PWAs won’t be able to benefit from app store traffic. Although they’re recognised as apps by search engines, they can’t be downloaded from an app store.

In this article, we have learned what a PWA is, why and when we need it, as well as its drawbacks. In the second and final part of this article series, we will learn how to build a simple PWA. **END** 🐧

Reference

[1] <https://en.wikipedia.org>

By: Neetesh Mehrotra

The author works at NIIT Technologies as a senior test engineer. His areas of interest are Java development and automation testing.

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU